

.NET AI Capability Exercise

Participant Guide — Build & Grow Your AI Workflow

A hands-on exercise that every developer in the .NET competency completes to strengthen and show how they work with AI. You'll get a feedback report and a personalized growth path — this is for development, not a graded test.

1. What This Is	3
2. Who Takes Part	3
3. Time and Effort	3
4. What You Get Out of It	3
5. How the Exercise is Structured	4
Part A: AI Workflow Foundation	4
Part B: Full-Stack Mini Project	4
Project Option 1: Backend-Heavy — Support Ticket Management System	5
Project Option 2: Frontend-Heavy — AI Learning Dashboard / Project Tracker	6
Project Option 3: .NET Full-Stack — SME ERP (Inventory Management)	8
Required Repository Structure	13
Part C Submission Templates	14
6. Tool-Specific Expectations	17
7. What Counts as Complete	17
8. How to Take Part	17
9. What Good Looks Like	17
10. Your Growth Path	17
11. Summary	18

1. What This Is

This is a hands-on capability exercise to help you develop — and make visible — how you use AI tools effectively, responsibly, and practically across the software development lifecycle. Everyone in the competency takes part; it is a shared part of how we build AI capability, not something a few people are singled out for.

It is not a graded test. You will build a small full-stack project and show your thinking, and in return, you get a feedback report and a clear sense of what to grow next. What matters is not only whether the final app works, but how you used AI for requirement analysis, planning, implementation, testing, debugging, code review, documentation, and reflection.

Making your thinking visible is the point.

2. Who Takes Part

All developers in the competency, from SE to TL level, working with stacks such as **.NET, or full-stack combinations**. It is common across roles and stacks, though the project option you choose may emphasize either backend or frontend depth. Because everyone does it, there is a shared baseline, and no one is measured against a different bar than their peers.

3. Time and Effort

The exercise is self-paced and meant to be completed within one week. You may work in any order; there is no required day-wise plan. Share your work by the agreed date.

- **Expected effort:** The mandatory Core of the project is scoped for roughly 8–12 focused hours.
- **Lifecycle Artifacts:** The rest of the week goes into the lifecycle artifacts — requirement analysis, prompt history, testing and debugging notes, and reflection, which are the main things the feedback looks at.

Important: Do not expand the Core application at the expense of these artifacts.

4. What You Get Out of It

You receive a feedback report detailing your strengths, your growth areas, and concrete next steps for developing your AI workflow. The report also gives a sense of where you currently sit in the AI capability framework and what would move you forward — think of it as a snapshot and a direction, not a grade. Your competency owner uses this as one input into your normal growth track.

Feedback focuses on areas like requirement analysis, prompting and context-setting, tool workflow, full-stack design, code quality, database design, testing depth, debugging, code review, documentation, ownership, and responsible AI judgment.

5. How the Exercise is Structured

It has three parts:

Part	Focus	Emphasis
Part A	AI Workflow Foundation	20%
Part B	Full-Stack Mini Project (Core + optional Stretch)	60%
Part C	Submission and Reflection	20%

Part C is completed through the participation form (see “How to Take Part”), where you share your repository and answer a few questions about your work. The percentages show where to put your effort — they are not exam weights.

Part A: AI Workflow Foundation

Objective

Show that you understand how AI should be used in a practical engineering workflow — thoughtfully, not as a simple code-generation shortcut.

Expected Submission

Submit a document named `tool-workflow.MD` covering:

1. Primary AI tool used.
2. How do you provide project context to the tool
3. How you use AI for requirement analysis, planning/design, code generation, validation, testing, debugging, and code review.
4. What information do you avoid sharing unnecessarily with AI tools?
5. How would you reuse this workflow in a real project?

Part B: Full-Stack Mini Project

Objective

Demonstrate practical AI-assisted delivery through a realistic full-stack assignment. Each project is split into a mandatory Core and an optional Stretch tier. Core is a small, buildable application that serves as a vehicle for showing your AI workflow. Stretch is optional and shows more advanced practice.

A clean, well-documented Core alone is a strong result. Both options are looked at the same way; the difference is the depth of evidence you show.

Common Technical Requirements

Whichever option you choose, every submission must include:

- Frontend application
- Backend API
- Database persistence
- Database setup or migration scripts
- Seed or sample data
- Input validation

- Error handling
- One working search or filter capability (Core); more in Stretch
- At least one meaningful test tier (Core); more in Stretch
- README setup instructions
- Full prompt history
- All planning, design, testing, debugging, review, reflection, and PR artifacts in the repository structure

The full set of lifecycle artifacts is required regardless of the option. Only the application surface area is small — the artifacts are the point.

Database Requirement

A database is mandatory. Acceptable options include PostgreSQL, MySQL, MongoDB, SQLite, H2, or any framework-supported local database. Provide your database choice, setup instructions, a schema/migration/initialization script, seed data, an environment variable example (if applicable), and steps to run locally.

Authentication

Authentication is optional — the focus is AI-assisted engineering across the lifecycle, not auth. If implemented well, it counts as Stretch evidence (login/logout, JWT or session auth, role-based access, protected routes, API authorization).

Project Option 1: Backend-Heavy — Support Ticket Management System

Business Context

A small application for managing support tickets. Internal users create, update, comment on, search, and progress tickets through a defined lifecycle.

Core (Mandatory)

Entities

User (seeded only – no user-management UI required)
 - id, name, email, role

Ticket
 - id, title, description, priority, status, assignedTo, createdBy, createdAt, updatedAt

Comment
 - id, ticketId, message, createdBy, createdAt

Features

- Create a ticket.
- List tickets.
- View ticket detail.
- Update ticket fields (title, description, priority, assignee).
- Change ticket status through the enforced state machine.
- Add comments to a ticket.
- Keyword search and filter by status.

- Persist all data; data survives restart.
- Validate required fields; reject invalid input at the backend.
- Show meaningful error states in the UI.

Status state machine *(the signature judgment piece — kept in Core)*

```
Open      -> In Progress
In Progress -> Resolved
Resolved  -> Closed
Open      -> Cancelled
In Progress -> Cancelled
```

Invalid transitions must be rejected by the backend and handled clearly in the frontend. This is deliberately the hardest part of Core because it is where engineering judgment shows.

Mandatory test tier: integration tests that prove the state-machine rules — valid transitions succeed, invalid transitions are rejected.

Stretch (Optional — evidence toward C1.1)

- Third entity or richer data model
- Full user CRUD and role management
- Authentication, protected routes, API authorization checks
- Filter by priority and assignee; sorting; pagination
- Additional test tiers: unit tests and edge-case/failure tests
- API documentation (Swagger / OpenAPI)
- Docker setup, CI workflow
- Reusable prompt templates, rules, or specs (persistent project context)

Core Acceptance Criteria

1. A user can create a ticket via the UI.
2. A user can view all tickets from the database.
3. A user can open a ticket detail view.
4. A user can update ticket fields and reassign.
5. A user can add comments.
6. Status changes only through valid transitions; invalid ones are rejected.
7. Keyword search and status filter work.
8. Data remains available after restart.
9. Backend validation prevents invalid records.
10. No secrets committed to the repo.
11. State-machine integration tests pass.

Required Repository Structure

Submit a Git repository following this structure as closely as possible:

```
ai-practical-assessment/
├─ README.md
├─ candidate-info.md
├─ tool-workflow.md
├─ requirements-analysis.md
```

```

├─ acceptance-criteria.md
├─ implementation-plan.md
├─ design-notes.md
├─ api-contract.md
├─ data-model.md
├─ ui-flow.md
├─ test-strategy.md
├─ test-results.md
├─ debugging-notes.md
├─ code-review-notes.md
├─ review-fixes.md
├─ pr-description.md
├─ reflection.md
├─ final-ai-usage-summary.md
├─ src/
├─ tests/
├─ database/
│   └─ schema-or-migrations/
│   └─ seed-data/
│   └─ setup-notes.md
├─ ai-prompts/
│   └─ planning.md
│   └─ design.md
│   └─ implementation.md
│   └─ testing.md
│   └─ debugging.md
│   └─ code-review.md
│   └─ documentation.md
├─ tool-specific/
│   └─ kiro-specs/
│   └─ cursor-workflow/
│   └─ other-tool-workflow/

```

Fill only the tool-specific folder relevant to your selected tool.

Part C Submission Templates

Use these as starting structures for the required artifacts — they are a floor, not a limit.

Candidate Information

File: candidate-info.md

```

# Candidate Information
Name: / Role: / Primary Technology Stack:
Primary AI Tool Used: / Project Option Selected:
Assessment Start Date: / Submission Date:
## Project Summary / ## Tools Used / ## Setup Summary

```

Requirement Analysis

File: requirements-analysis.md

```

# Requirement Analysis

```

```
## Selected Project Option
## My Understanding (in your own words)
## Functional Requirements / ## Non-Functional Requirements
## Assumptions / ## Clarifications (questions for a product owner)
## Edge Cases
```

Acceptance Criteria

File: acceptance-criteria.md

```
# Acceptance Criteria
## Core - [ ] ...
## Validation - [ ] ...
## Error Handling - [ ] ...
## Testing - [ ] ...
## Documentation - [ ] ...
```

Implementation Plan

File: implementation-plan.md

```
# Implementation Plan
## Overview / ## Task Breakdown / ## Milestones
## AI Usage Plan / ## Risks / ## Mitigation
```

Design Notes

File: design-notes.md

```
# Design Notes
## Architecture Overview (frontend, backend, database)
## Frontend Design / ## Backend Design / ## Database Design
## Validation Strategy / ## Error Handling Strategy
## Testing Strategy Link
```

API Contract

File: api-contract.md

```
# API Contract
## Endpoint: Method: Path: Purpose:
### Request { } ### Response { }
### Validation Rules ### Error Responses
```

Test Strategy

File: test-strategy.md

```
# Test Strategy
## Test Scope
## Unit Tests / ## Component Tests
## API / Integration Tests / ## Edge Case Tests
## Tests Not Covered (and why)
```

Debugging Notes

File: debugging-notes.md

```
# Debugging Notes
## Issue 1
### Problem / ### How I Investigated
### How AI Helped / ### What I Validated / ### Final Fix
```


Code Review Notes

File: code-review-notes.md

```
# Code Review Notes
## AI-Assisted Review Summary / ## My Review Observations
## Changes Made After Review / ## Suggestions Rejected (and why)
```

Reflection

File: reflection.md

```
# Reflection
## What I Built / ## How I Used AI (across the lifecycle)
## What AI Helped With Most / ## What AI Got Wrong
## How I Validated AI Output / ## What I Would Improve Next
## Reusable Workflow (prompts, rules, specs, templates)
```

PR Description

File: pr-description.md

```
# PR Description
## Summary / ## Features Implemented / ## Technical Changes
## Database Changes / ## Testing Done / ## AI Usage Summary
## Screenshots / Demo Notes / ## Known Limitations / ## Future Improvements
```

Prompt History Expectations

Group prompt history by activity under [ai-prompts/](#). For each prompt, capture: prompt text or summary, AI response summary, what you accepted, changed, or rejected, and why.

- **Strong history shows:** Context setting, continuous iteration, stack-specific guidance, and validation prompts.
- **Weak history shows:** One-line prompts only, no iteration, and copying answers blindly.

6. Tool-Specific Expectations

Kiro: Submit [tool-specific/kiro-specs/](#) containing [requirements.md](#), [design.md](#), and [tasks.md](#). Show you used Kiro as a spec-driven tool rather than just a basic snippet generator.

Cursor: Submit [tool-specific/cursor-workflow/](#) with [project-context.md](#), [spec.md](#), [tasks.md](#), and [cursor-rules-or-instructions.md](#). Show persistent context and traceability.

Claude / Other: Submit [tool-specific/other-tool-workflow/](#) containing equivalent structural context logs and usage notes.

Note: *Your choice of tool does not disadvantage you. Stronger work shows reusable, persistent project context and standards enforced through your tool of choice — not a specific product's artifacts.*

7. What Counts as Complete

To ensure your feedback is useful, your submission should include a working frontend, working backend, database persistence, database setup/migration files, seed data, README setup instructions, basic tests, prompt history, requirement analysis, design notes, reflection, and a PR description. Missing elements redirect the growth pointer focus toward filling those exact workflow gaps.

8. How to Take Part

1. Once your project is ready, share it through the single online submission form. There is no review call to book and nothing to host or deploy.
2. Provide a link to your Git repository, your chosen option, and short written answers explaining your key engineering decisions and trade-offs in your own words.
3. Keep your commit history organized, as some questions ask you to point to specific places in your repo (e.g., where you fixed an AI mistake).

9. What Good Looks Like

The feedback weighs how you used AI across the lifecycle and how well you understand and own your solution — not just whether the app runs.

- **Strong work shows:** Clear problem understanding before coding, well-documented iterations, clean architecture, automated tests, and honest reflections.
- **Weaker work shows:** Direct copy-pasting with little understanding, shallow prompt history, broken configurations, and generic documentation.

In short, the journey and your understanding matter as much as the final code. Make your thinking visible.

10. Your Growth Path

Feedback provides direct markers to advance your current lifecycle execution:

- **Building the basics:** Focus on requirement gathering and context-setting.
- **Solid and growing:** Focus on deepening test matrices and code review maturity.
- **Strong across the lifecycle:** Focus on advanced edge-testing and reusable asset configurations.
- **Ready to lead:** Mature workflows with reusable prompts, rules, or specs — ready to mentor others.

Your competency owner uses the report and this direction within your normal growth conversations — it informs where you are in the capability framework rather than stamping a grade on you.

11. Summary

You'll complete a self-paced, one-week AI-assisted full-stack exercise using an approved AI tool (Cursor, Kiro, Claude, or equivalent). Choose one project option — Backend-heavy

(Support Ticket Management System), Frontend-heavy (AI Learning Dashboard / Project Tracker), or .NET Full-Stack (SME ERP: Inventory Management) — and build the mandatory Core; Stretch is optional.

Your project should include a frontend, a backend API, database persistence, setup/migration files, seed data, validation, error handling, search/filter, tests, README instructions, AI prompt/workflow history, a reflection, and a PR description. Use AI thoughtfully, make your thinking visible, and submit your repository link through the evaluation form when complete. Everyone in the competency does this — it's a shared step in building how we all work with AI.